Компьютерная графика

Лекция 6: blending, stencil buffer, render to texture, пост-обработка

2026

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты

Полупрозрачность



Full transparent window



Partially transparent window

Полупрозрачность



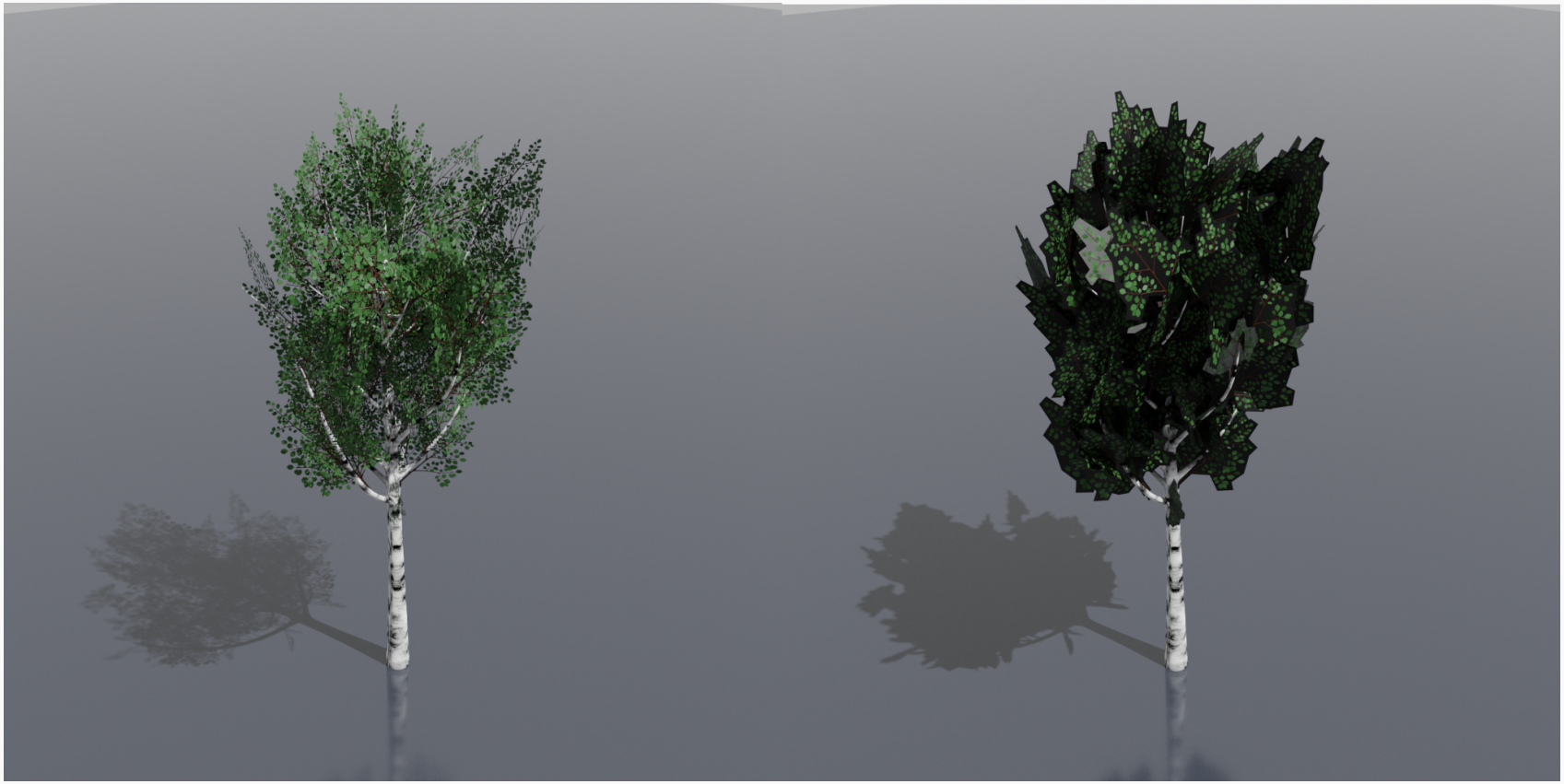
Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность

Деревья



Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность

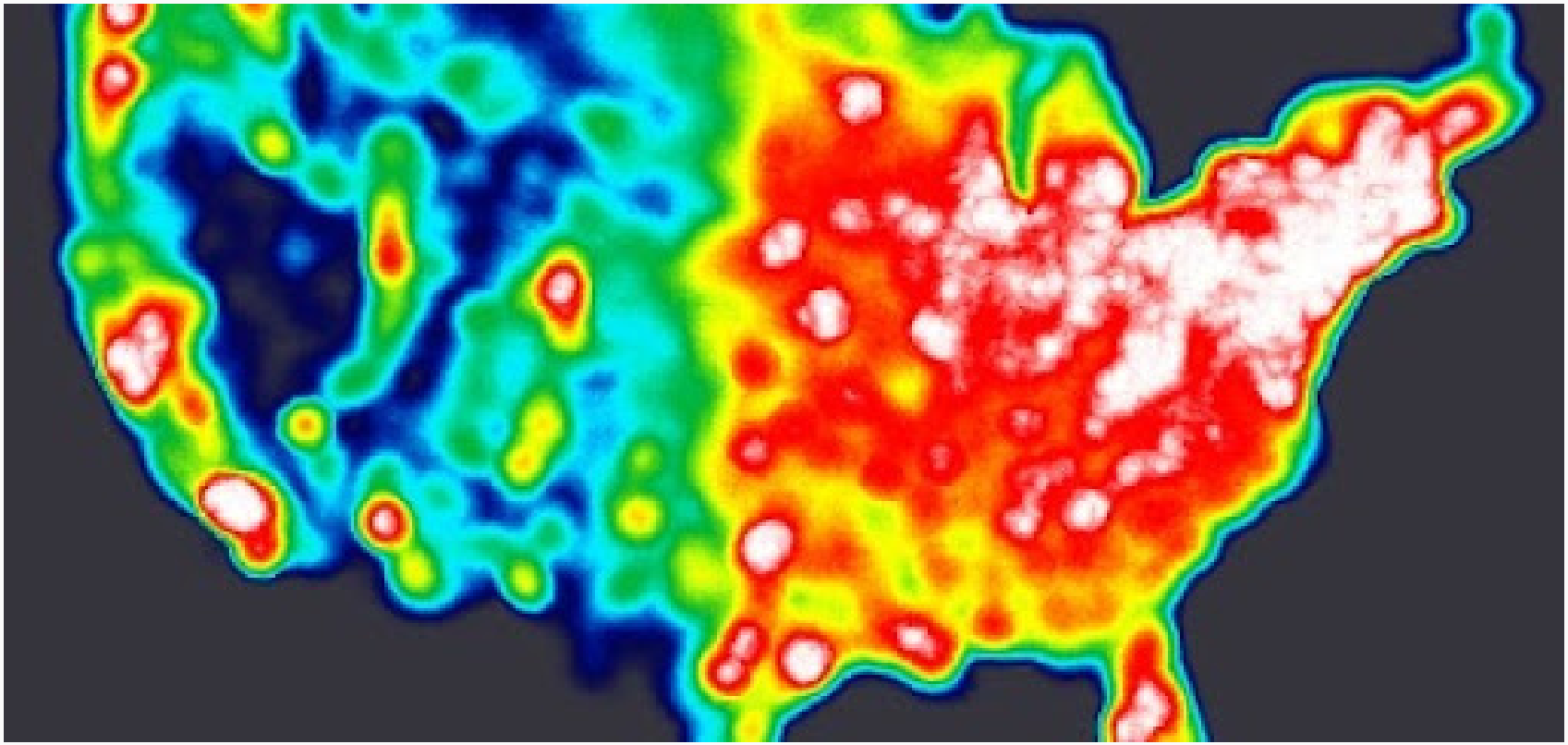
Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность
  - Алгоритмы освещения и теней

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность
  - Алгоритмы освещения и теней
  - Heatmaps

Heatmap



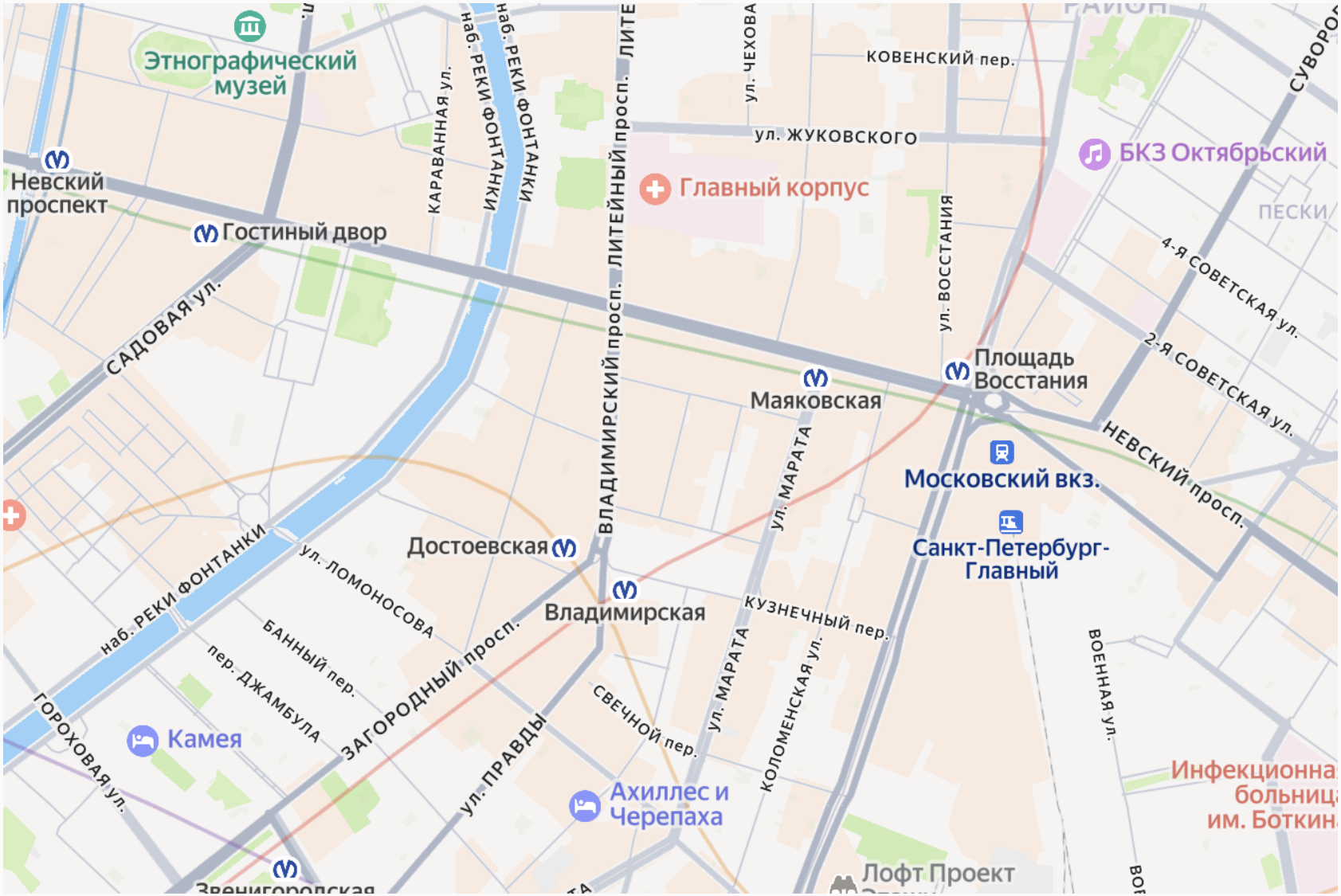
Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность
  - Алгоритмы освещения и теней
  - Heatmaps

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность
  - Алгоритмы освещения и теней
  - Heatmaps
  - Ручное сглаживание

Yandex maps


Этнографический музей
Невский проспект
Гостиный двор
САДОВАЯ ул.
КАРАВАННАЯ ул.
наб. РЕКИ ФОНТАНКИ
ВЛАДИМИРСКИЙ просп.
ЛИТЕЙНЫЙ просп.
ул. ЧЕХОВА
КОВЕНСКИЙ пер.
ул. ЖУКОВСКОГО
Главный корпус
БКЗ Октябрьский
ПЕСКИ
ул. ВОССТАНИЯ
4-Я СОВЕТСКАЯ ул.
2-Я СОВЕТСКАЯ ул.
Площадь Восстания
Маяковская
НЕВСКИЙ просп.
Московский вкз.
Санкт-Петербург-Главный
ул. МАРАТА
Достоевская
Владимирская
наб. РЕКИ ФОНТАНКИ
ул. ЛОМОНОСОВА
БАННЫЙ пер.
пер. ДЖАМБУЛА
ГОРОХОВАЯ ул.
ЗАГОРОДНЫЙ просп.
КУЗНЕЧНЫЙ пер.
СВЕЧНОЙ пер.
ул. ПРАВДЫ
КОЛОМЕНСКАЯ ул.
ВОЕННАЯ ул.
Камея
Ахиллес и Черепаха
Инфекционная больница им. Боткин
Лофт Проект
Звенигородская

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность
  - Алгоритмы освещения и теней
  - Heatmaps
  - Ручное сглаживание

Blending (смешивание)

- По умолчанию результат фрагментного шейдера (`@location(0) vec4f`) записывается в пиксель экрана, стирая то, что было в нём до этого
- Иногда мы хотим ‘смешать’ результат и пиксель экрана, и записать результат смешивания:
  - Полупрозрачные объекты
  - Растительность
  - Алгоритмы освещения и теней
  - Heatmaps
  - Ручное сглаживание
  - И т.д.

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`
- Уравнение blending'а:

$$dst \leftarrow f(C_{src} \cdot src, C_{dst} \cdot dst)$$

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`
- Уравнение blending'а:

$$dst \leftarrow f(C_{src} \cdot src, C_{dst} \cdot dst)$$

  - src – результат фрагментного шейдера

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`
- Уравнение blending'а:

$$dst \leftarrow f(C_{src} \cdot src, C_{dst} \cdot dst)$$

  - *src* – результат фрагментного шейдера
  - *dst* – пиксель на экране

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`
- Уравнение blending'а:

$$dst \leftarrow f(C_{src} \cdot src, C_{dst} \cdot dst)$$

  - src – результат фрагментного шейдера
  - dst – пиксель на экране
  - f – настраиваемая функция

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`
- Уравнение blending'а:

$$dst \leftarrow f(C_{src} \cdot src, C_{dst} \cdot dst)$$

  - src – результат фрагментного шейдера
  - dst – пиксель на экране
  - f – настраиваемая функция
  - C_{src} и C_{dst} – настраиваемые веса

Blending (смешивание)

- Настраивается в `color_target_state.blend` в `render_pipeline_descriptor.fragment.targets`
- Уравнение blending'а:

$$dst \leftarrow f(C_{src} \cdot src, C_{dst} \cdot dst)$$

  - src – результат фрагментного шейдера
  - dst – пиксель на экране
  - f – настраиваемая функция
  - C_{src} и C_{dst} – настраиваемые веса
  - Настраивается отдельно для RGB и для A компонент

Blending (смешивание)

- Настройка функции f: *blend operation*:
  - `Add` $f(src, dst) = src + dst$
  - `Subtract` $f(src, dst) = src - dst$
  - `ReverseSubtract` $f(src, dst) = dst - src$
  - `Min` $f(src, dst) = min(src, dst)$
  - `Max` $f(src, dst) = max(src, dst)$

Blending (смешивание)

- Настройка весов C_{src} и C_{dst}: *blend factor*
  - `Zero`: $C = 0$
  - `One`: $C = 1$
  - `Src`: $C = src$ (покомпонентно)
  - `OneMinusSrc`: $C = 1 - src$ (покомпонентно)
  - `srcAlpha`: $C = src_A$
  - `OneMinusSrcAlpha`: $C = 1 - src_A$
  - И т.д.

Blending (смешивание)

- Обычно $src = (R, G, B, A)$ означает, что цвет имеет полупрозрачность (точнее – *непрозрачность*, или *opacity*) равную A
  - $A = 0$ – полностью прозрачный цвет
  - $A = 1$ – полностью непрозрачный цвет

Blending (смешивание)

- Обычно $src = (R, G, B, A)$ означает, что цвет имеет полупрозрачность (точнее – *непрозрачность*, или *opacity*) равную A
  - $A = 0$ – полностью прозрачный цвет
  - $A = 1$ – полностью непрозрачный цвет
- Для этого нужна такая формула смешивания:

$$dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst = \mathbf{lerp}(dst, src, src_A)$$

Blending (смешивание)

- Обычно $src = (R, G, B, A)$ означает, что цвет имеет полупрозрачность (точнее – *непрозрачность*, или *opacity*) равную A
  - $A = 0$ – полностью прозрачный цвет
  - $A = 1$ – полностью непрозрачный цвет
- Для этого нужна такая формула смешивания:

$$dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst = \mathsf{lerp}(dst, src, src_A)$$

- Этому соответствует настройка

```
BlendComponent {
  operation: Add,
  src_factor: SrcAlpha,
  dst_factor: OneMinusSrcAlpha,
}
```

Blending (смешивание)

- Обычно $src = (R, G, B, A)$ означает, что цвет имеет полупрозрачность (точнее – *непрозрачность*, или *opacity*) равную A
  - $A = 0$ – полностью прозрачный цвет
  - $A = 1$ – полностью непрозрачный цвет
- Для этого нужна такая формула смешивания:

$$dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst = \mathsf{lerp}(dst, src, src_A)$$

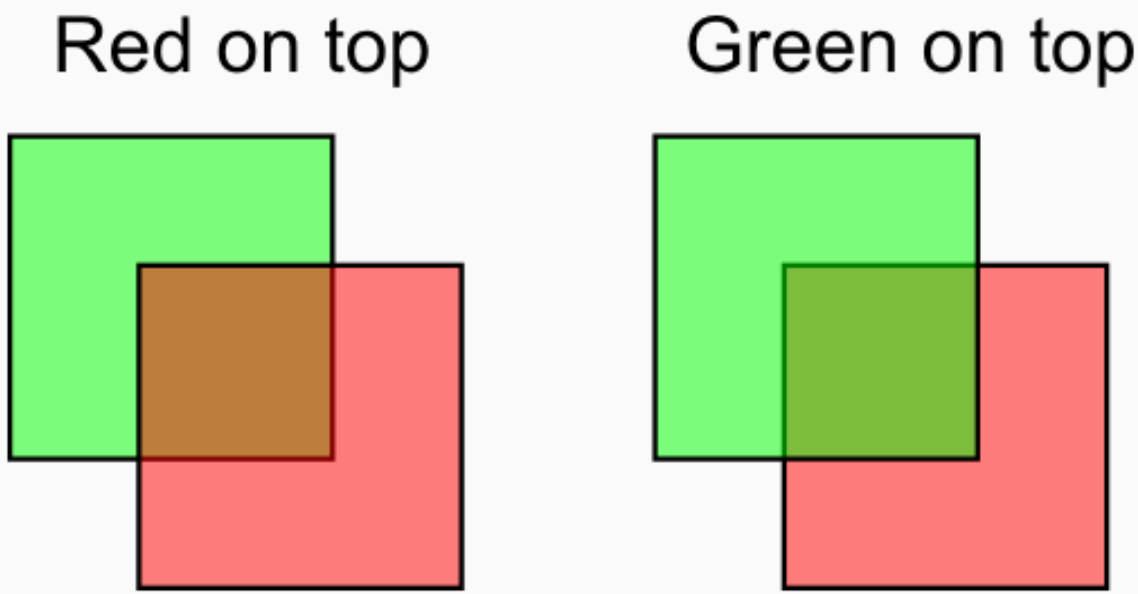
- Этому соответствует настройка

```
BlendComponent {
  operation: Add,
  src_factor: SrcAlpha,
  dst_factor: OneMinusSrcAlpha,
}
```

- Это *один из самых типичных* способов использовать blending (но не самый правильный)

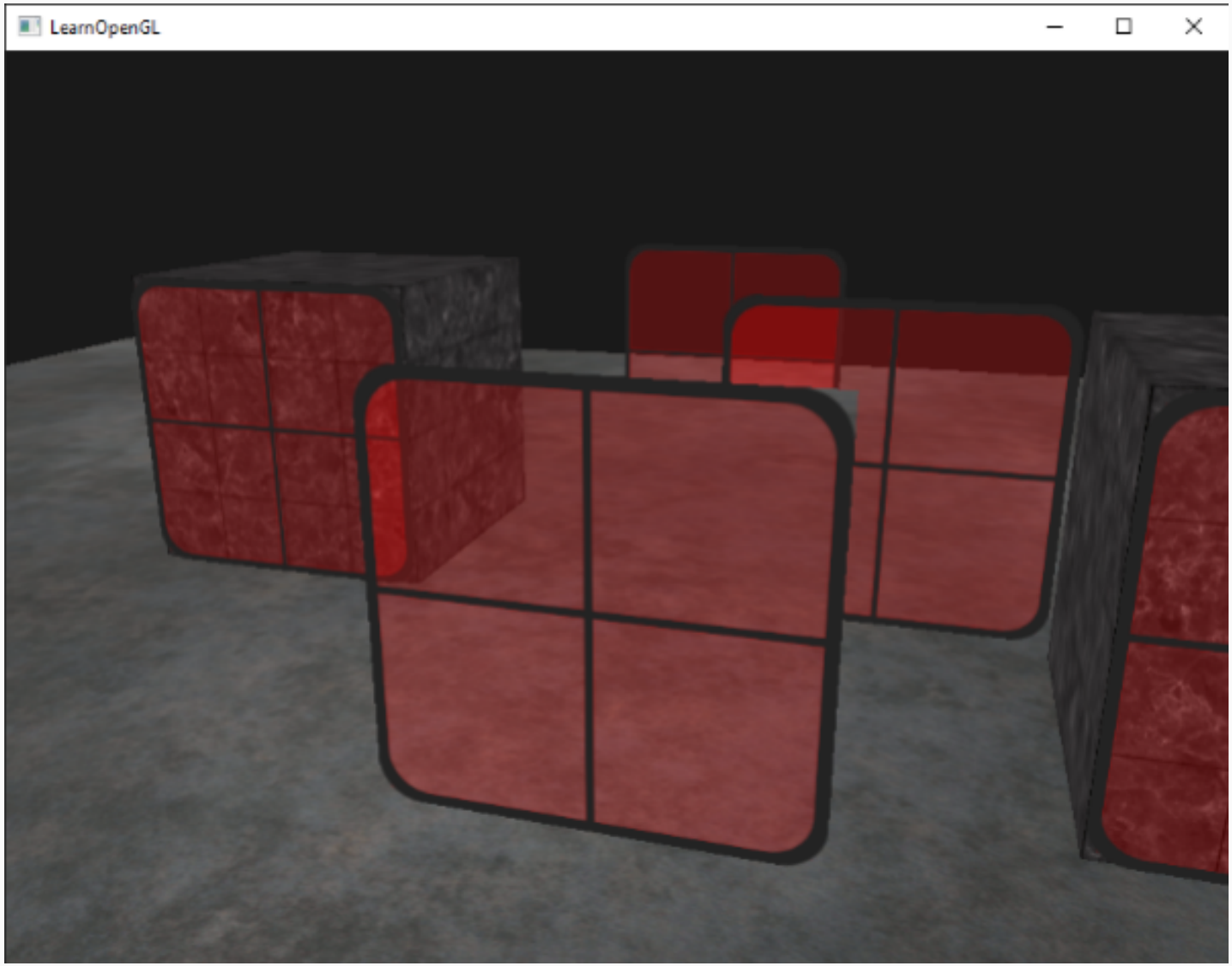
Blending (смешивание)

- Такое смешивание некоммутативно, т.е. результат зависит от порядка рисования
- Бывают конкретные ситуации, когда коммутативность не нарушается (наложение двух объектов с `A = 0.5` или наложение нескольких объектов одинакового цвета)



Blending (смешивание)

- Смешивание некорректно работает с тестом глубины



Blending (смешивание)

- В общем случае нужно сортировать полупрозрачные объекты и рисовать в порядке уменьшения расстояния до камеры

Blending (смешивание)

- В общем случае нужно сортировать полупрозрачные объекты и рисовать в порядке уменьшения расстояния до камеры
  - Что именно считать расстоянием до камеры (расстояние от центра объекта / от ближайшей вершины / от самой далёкой вершины / среднее между минимальным и максимальным расстоянием) – большой вопрос

Blending (смешивание)

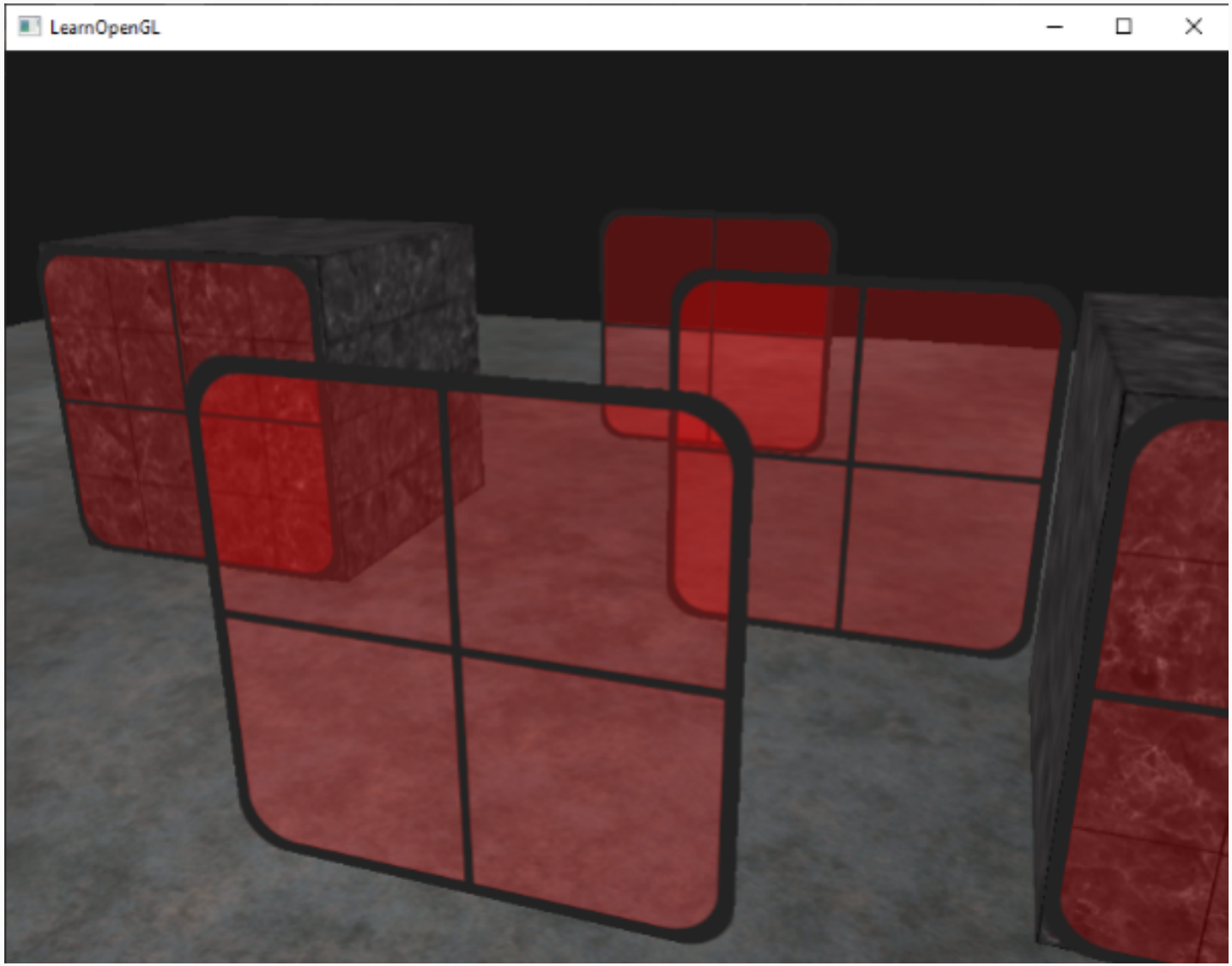
- В общем случае нужно сортировать полупрозрачные объекты и рисовать в порядке уменьшения расстояния до камеры
  - Что именно считать расстоянием до камеры (расстояние от центра объекта / от ближайшей вершины / от самой далёкой вершины / среднее между минимальным и максимальным расстоянием) – большой вопрос
  - Для любого варианта можно найти примеры, ломающие его (цикл между объектами, накрывающими друг друга)

Blending (смешивание)

- В общем случае нужно сортировать полупрозрачные объекты и рисовать в порядке уменьшения расстояния до камеры
  - Что именно считать расстоянием до камеры (расстояние от центра объекта / от ближайшей вершины / от самой далёкой вершины / среднее между минимальным и максимальным расстоянием) – большой вопрос
  - Для любого варианта можно найти примеры, ломающие его (цикл между объектами, накрывающими друг друга)
- Order-independent transparency (OIT) – активная тема исследований

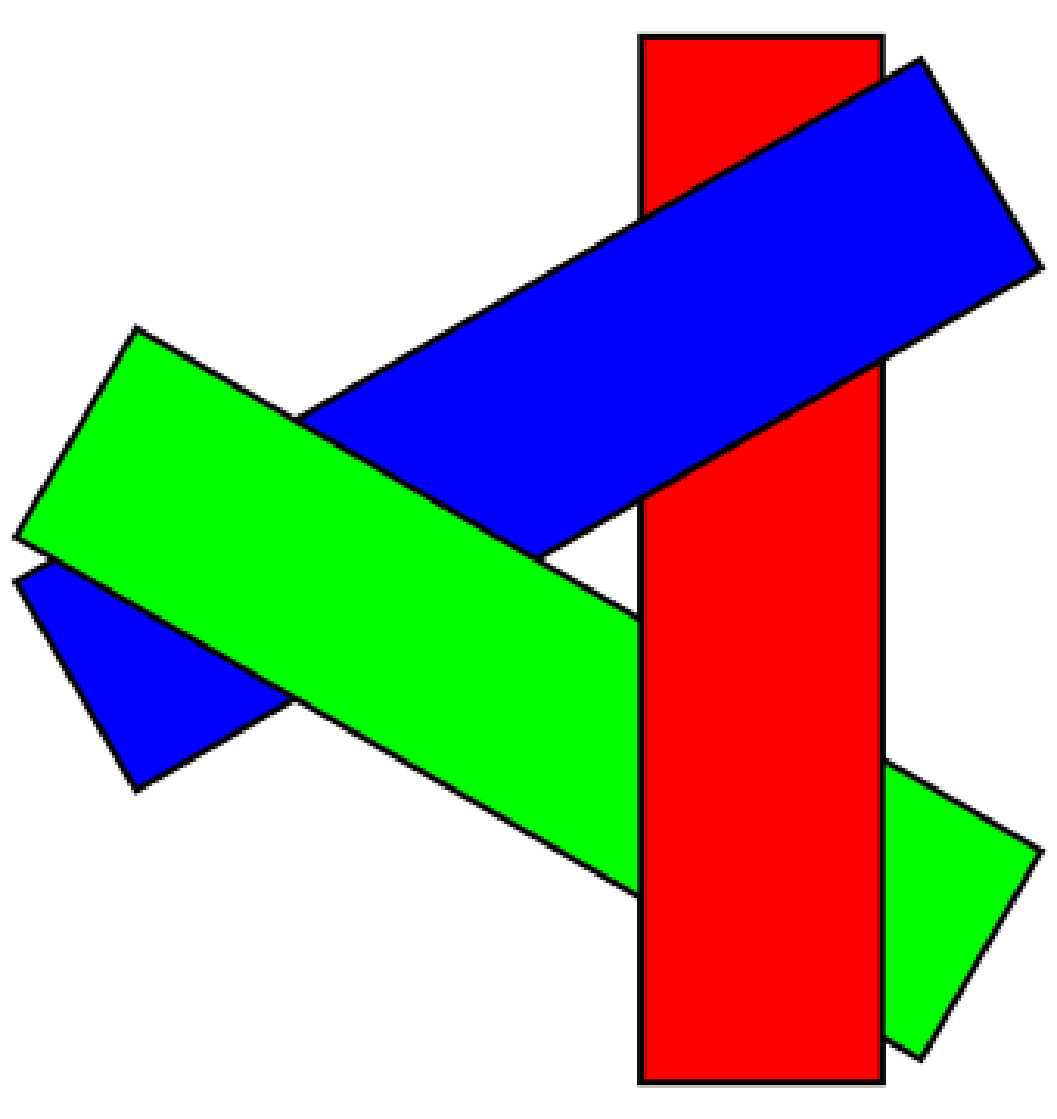
Blending (смешивание)

- Сортировка по расстоянию до камеры



Blending (смешивание)

- Случай, когда сортировка не поможет



Аддитивное смешивание

- Один часто используемый вариант смешивания – аддитивное смешивание
- $dst \leftarrow src + dst$ или $dst \leftarrow src_A \cdot src + dst$

Аддитивное смешивание

- Один часто используемый вариант смешивания – аддитивное смешивание
- $dst \leftarrow src + dst$ или $dst \leftarrow src_A \cdot src + dst$

```
BlendComponent {
  operation: Add,
  src_factor: SrcAlpha, // или One
  dst_factor: One,
}
```

Аддитивное смешивание

- Один часто используемый вариант смешивания – аддитивное смешивание
- $dst \leftarrow src + dst$ или $dst \leftarrow src_A \cdot src + dst$

```
BlendComponent {
  operation: Add,
  src_factor: SrcAlpha, // или One
  dst_factor: One,
}
```

- Такое смешивание коммутативно $\Longrightarrow$ не важно, в каком порядке рисовать объекты

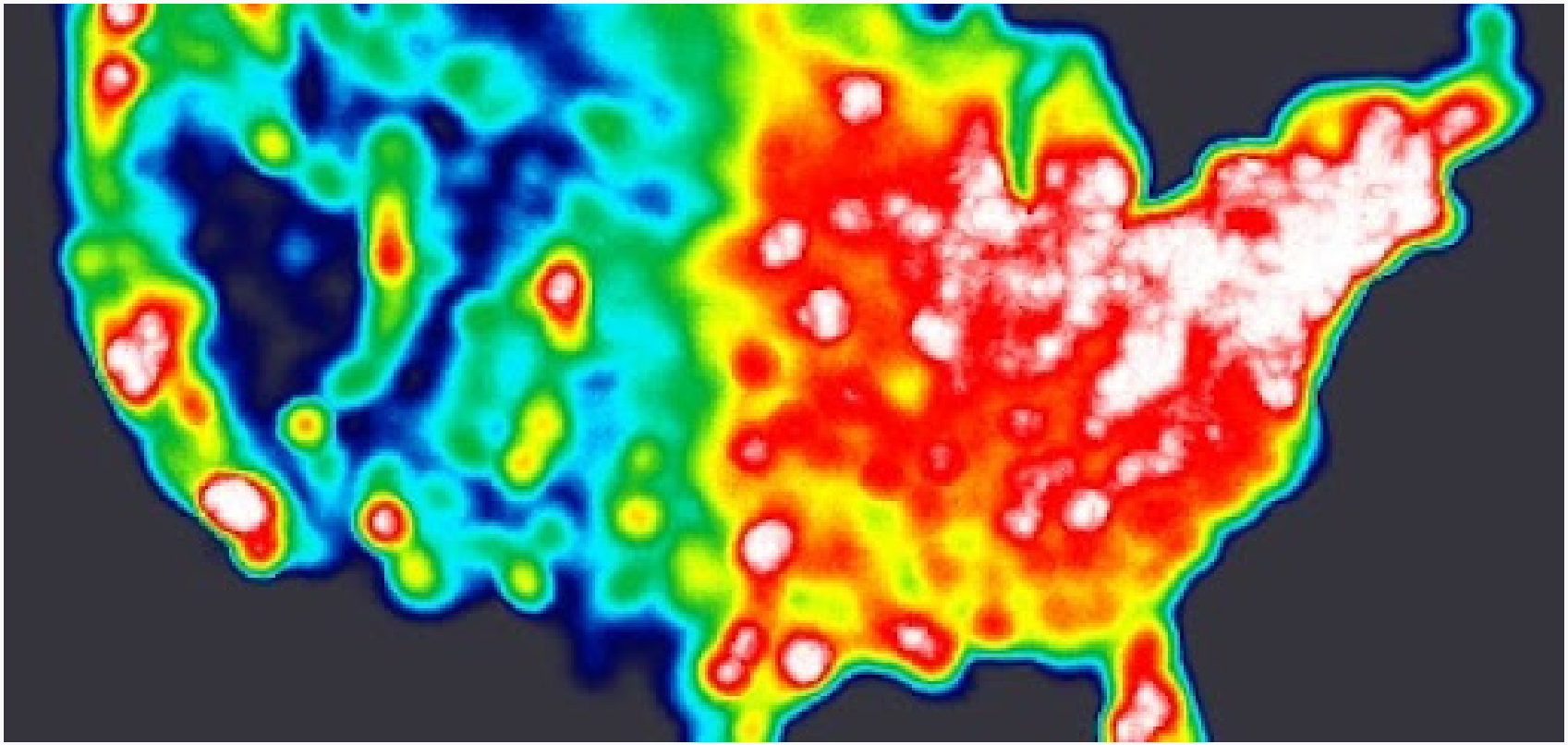
Аддитивное смешивание

- Один часто используемый вариант смешивания – аддитивное смешивание
- $dst \leftarrow src + dst$ или $dst \leftarrow src_A \cdot src + dst$

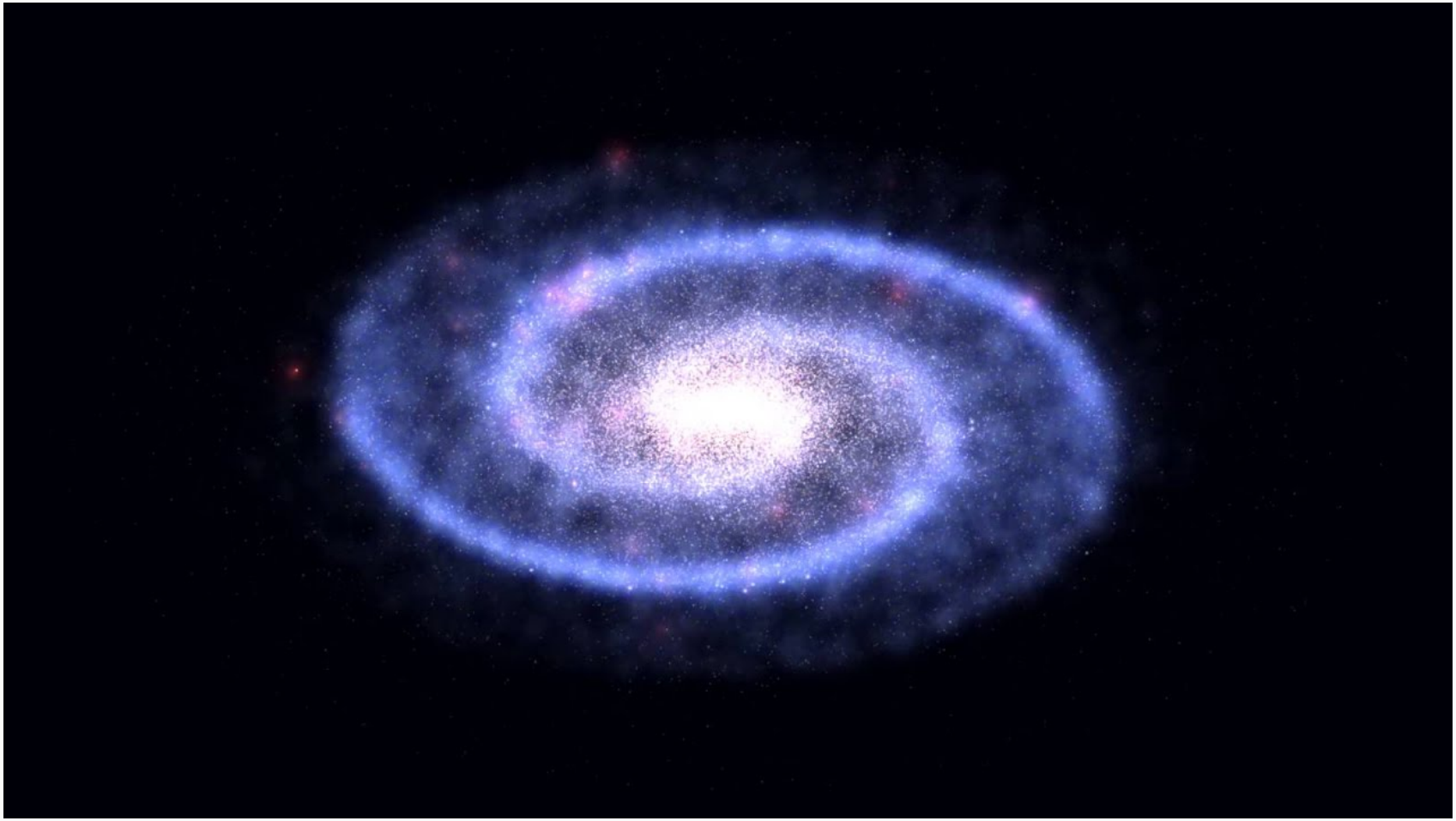
```
BlendComponent {
  operation: Add,
  src_factor: SrcAlpha, // или One
  dst_factor: One,
}
```

- Такое смешивание коммутативно $\Longrightarrow$ не важно, в каком порядке рисовать объекты
- Heatmaps, облака точек

Аддитивное смешивание



Аддитивное смешивание



Blending (смешивание)

- Формула $dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst$ работает только если destination – непрозрачный цвет

Blending (смешивание)

- Формула $dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst$ работает только если destination – непрозрачный цвет
- Если мы хотим наложить несколько полупрозрачных объектов, и получить суммарную полупрозрачную картинку, нужна более сложная формула

Blending (смешивание)

- Формула $dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst$ работает только если destination – непрозрачный цвет
- Если мы хотим наложить несколько полупрозрачных объектов, и получить суммарную полупрозрачную картинку, нужна более сложная формула
- Пусть есть цвет C_0, на который накладывается цвет C_1 с opacity a_1, а потом C_2 с opacity a_2: получится цвет

Blending (смешивание)

- Формула $dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst$ работает только если destination – непрозрачный цвет
- Если мы хотим наложить несколько полупрозрачных объектов, и получить суммарную полупрозрачную картинку, нужна более сложная формула
- Пусть есть цвет C_0, на который накладывается цвет C_1 с opacity a_1, а потом C_2 с opacity a_2: получится цвет

$$C_2 a_2 + C_1 a_1 (1 - a_2) + C_0 (1 - a_1)(1 - a_2)$$

Blending (смешивание)

- Формула $dst \leftarrow src_A \cdot src + (1 - src_A) \cdot dst$ работает только если destination – непрозрачный цвет
- Если мы хотим наложить несколько полупрозрачных объектов, и получить суммарную полупрозрачную картинку, нужна более сложная формула
- Пусть есть цвет C_0, на который накладывается цвет C_1 с opacity a_1, а потом C_2 с opacity a_2: получится цвет

$$C_2 a_2 + C_1 a_1 (1 - a_2) + C_0 (1 - a_1)(1 - a_2)$$

- Мы хотим привести это к виду $C' a' + C_0 (1 - a')$

Blending (смешивание)

- Мы хотим привести это к виду $C'a' + C_0(1 - a')$

Blending (смешивание)

- Мы хотим привести это к виду $C'a' + C_0(1 - a')$
- Отсюда

$$a' = 1 - (1 - a_1)(1 - a_2) = a_1 + a_2 - a_1a_2 = a_2 + a_1(1 - a_2)$$

$$C' = \frac{C_2a_2 + C_1a_1(1-a_2)}{a_2 + a_1(1-a_2)}$$

Blending (смешивание)

- Мы хотим привести это к виду $C'a' + C_0(1 - a')$
- Отсюда

$$a' = 1 - (1 - a_1)(1 - a_2) = a_1 + a_2 - a_1 a_2 = a_2 + a_1(1 - a_2)$$

$$C' = \frac{C_2 a_2 + C_1 a_1 (1 - a_2)}{a_2 + a_1 (1 - a_2)}$$

- Это т.н. *over operator* смешивания (см. Porter, Duff'84)

Blending (смешивание)

- Мы хотим привести это к виду $C'a' + C_0(1 - a')$
- Отсюда

$$a' = 1 - (1 - a_1)(1 - a_2) = a_1 + a_2 - a_1 a_2 = a_2 + a_1(1 - a_2)$$

$$C' = \frac{C_2 a_2 + C_1 a_1 (1 - a_2)}{a_2 + a_1 (1 - a_2)}$$

- Это т.н. *over operator* смешивания (см. Porter, Duff'84)
- Такую функцию **не выразить** через стандартный блендинг: он не умеет разделить результирующий цвет на его альфа-канал

Blending (смешивание)

- Мы хотим привести это к виду $C'a' + C_0(1 - a')$
- Отсюда

$$a' = 1 - (1 - a_1)(1 - a_2) = a_1 + a_2 - a_1 a_2 = a_2 + a_1(1 - a_2)$$

$$C' = \frac{C_2 a_2 + C_1 a_1 (1 - a_2)}{a_2 + a_1 (1 - a_2)}$$

- Это т.н. *over operator* смешивания (см. Porter, Duff'84)
- Такую функцию **не выразить** через стандартный блендинг: он не умеет разделить результирующий цвет на его альфа-канал
- $\Longrightarrow$ Решение: *премультиплицированный* альфа-канал

Premultiplied alpha

- Идея: давайте хранить цвета не в виде (R, G, B, A), а в виде $(R \cdot A, G \cdot A, B \cdot A, A)$

Premultiplied alpha

- Идея: давайте хранить цвета не в виде (R, G, B, A), а в виде $(R \cdot A, G \cdot A, B \cdot A, A)$
- Тогда формула blending'а заменится на

$$a' = a_2 + a_1(1 - a_2)$$

$$C' \cdot a' = (C_2 \cdot a_2) + (C_1 \cdot a_1)(1 - a_2)$$

Premultiplied alpha

- Идея: давайте хранить цвета не в виде (R, G, B, A), а в виде $(R \cdot A, G \cdot A, B \cdot A, A)$
- Тогда формула blending'а заменится на

$$a' = a_2 + a_1(1 - a_2)$$

$$C' \cdot a' = (C_2 \cdot a_2) + (C_1 \cdot a_1)(1 - a_2)$$

- $\Longrightarrow$ Если и входные, и выходные цвета премультиплицированные, то правильный blending **можно** выразить встроенными операциями:

```
BlendComponent {
  operation: Add,
  src_factor: One,
  dst_factor: OneMinusSrcAlpha,
}
```

Premultiplied alpha: mipmaps

- При обычной генерации mipmaps для полупрозрачных изображений есть проблема: очень прозрачные или полностью прозрачные пиксели дают такой же вклад в результат, как и непрозрачные

Premultiplied alpha: mipmaps

- При обычной генерации mipmaps для полупрозрачных изображений есть проблема: очень прозрачные или полностью прозрачные пиксели дают такой же вклад в результат, как и непрозрачные
- Пример: три чёрных прозрачных пикселя $(R, G, B, A) = (0, 0, 0, 0)$ усредняются с одним красным непрозрачным $(R, G, B, A) = (1, 0, 0, 1)$, и получается тёмно-красный пиксель с opacity 25%: $(0.25, 0, 0, 0.25)$

Premultiplied alpha: mipmaps

- При обычной генерации mipmaps для полупрозрачных изображений есть проблема: очень прозрачные или полностью прозрачные пиксели дают такой же вклад в результат, как и непрозрачные
- Пример: три чёрных прозрачных пикселя $(R, G, B, A) = (0, 0, 0, 0)$ усредняются с одним красным непрозрачным $(R, G, B, A) = (1, 0, 0, 1)$, и получается тёмно-красный пиксель с opacity 25%: $(0.25, 0, 0, 0.25)$
- Значение opacity верное, но значение цвета – нет: пиксель должен был остаться красным

Premultiplied alpha: mipmaps

- При обычной генерации mipmaps для полупрозрачных изображений есть проблема: очень прозрачные или полностью прозрачные пиксели дают такой же вклад в результат, как и непрозрачные
- Пример: три чёрных прозрачных пикселя $(R, G, B, A) = (0, 0, 0, 0)$ усредняются с одним красным непрозрачным $(R, G, B, A) = (1, 0, 0, 1)$, и получается тёмно-красный пиксель с opacity 25%: $(0.25, 0, 0, 0.25)$
- Значение opacity верное, но значение цвета – нет: пиксель должен был остаться красным
- Если хранить пиксели текстуры в премультиплицированном виде, проблема отпадает: цвета автоматически усредняются пропорционально значениям альфа-канала:

$$a' = \frac{1}{4}(a_0 + a_1 + a_2 + a_3)$$

$$C' \cdot a' = \frac{1}{4}(C_0 \cdot a_0 + C_1 \cdot a_1 + C_2 \cdot a_2 + C_3 \cdot a_3)$$

Premultiplied alpha

- У такого способа хранения текстур есть минус: значения цветовых каналов ограничены значением альфа-канала, и теряется их точность

Premultiplied alpha

- У такого способа хранения текстур есть минус: значения цветовых каналов ограничены значением альфа-канала, и теряется их точность
- Обычно это не проблема: если пиксель сильно прозрачный, то слабая точность его цвета всё равно слабо заметна

Premultiplied alpha

- У такого способа хранения текстур есть минус: значения цветовых каналов ограничены значением альфа-канала, и теряется их точность
- Обычно это не проблема: если пиксель сильно прозрачный, то слабая точность его цвета всё равно слабо заметна
- В общем случае, если вы делаете хоть что-то нетривиальное с полупрозрачностью, лучше **сразу использовать** премультиплицированные цвета

Blending (смешивание)

- khronos.org/opengl/wiki/Blending
- opengl-tutorial.org/intermediate-tutorials/tutorial-10-transparency
- learnopengl.com/Advanced-OpenGL/Blending
- Porter, Duff - Compositing Digital Images

Stencil buffer (буфер трафарета)

- Особый буфер, хранящий произвольные данные (почти всегда unsigned 8-bit на каждый пиксель), с особыми побитовыми операциями над ними

Stencil buffer (буфер трафарета)

- Особый буфер, хранящий произвольные данные (почти всегда unsigned 8-bit на каждый пиксель), с особыми побитовыми операциями над ними
- Чем-то аналогичен буферу глубины: тоже хранит какие-то данные, такого же размера (как и цветовой буфер), тоже позволяет рисовать или не рисовать пиксель по какому-то условию (stencil test – происходит *перед* depth test'ом)

Stencil buffer (буфер трафарета)

- Особый буфер, хранящий произвольные данные (почти всегда unsigned 8-bit на каждый пиксель), с особыми побитовыми операциями над ними
- Чем-то аналогичен буферу глубины: тоже хранит какие-то данные, такого же размера (как и цветовой буфер), тоже позволяет рисовать или не рисовать пиксель по какому-то условию (stencil test – происходит *перед* depth test'ом)
- Может быть отдельной текстурой в формате `Stencil8`, либо храниться вместе с буфером глубины в формате `Depth24PlusStencil8` или `Depth32FloatStencil8`

Stencil buffer (буфер трафарета)

- Особый буфер, хранящий произвольные данные (почти всегда unsigned 8-bit на каждый пиксель), с особыми побитовыми операциями над ними
- Чем-то аналогичен буферу глубины: тоже хранит какие-то данные, такого же размера (как и цветовой буфер), тоже позволяет рисовать или не рисовать пиксель по какому-то условию (stencil test – происходит *перед* depth test'ом)
- Может быть отдельной текстурой в формате `Stencil8`, либо храниться вместе с буфером глубины в формате `Depth24PlusStencil8` или `Depth32FloatStencil8`
- Подключается к render_pass'у так же, как буфер глубины – через `render_pass_descriptor.depth_stencil_attachment`

Stencil тест

- Stencil test настраивается в `render_pipeline_descriptor.depth_stencil.stencil`

Stencil тест

- Stencil test настраивается в `render_pipeline_descriptor.depth_stencil.stencil`
  - `read_mask`, `write_mask` – побитовые маски, которые будут применены к значениям в stencil buffer'е при его чтении/записи

Stencil тест

- Stencil test настраивается в `render_pipeline_descriptor.depth_stencil.stencil`
  - `read_mask`, `write_mask` – побитовые маски, которые будут применены к значениям в stencil buffer'е при его чтении/записи
  - `front`, `back` – поведение stencil test'а для front-facing и back-facing граней

Stencil тест

- Stencil test настраивается в `render_pipeline_descriptor.depth_stencil.stencil`
  - `read_mask`, `write_mask` – побитовые маски, которые будут применены к значениям в stencil buffer'е при его чтении/записи
  - `front`, `back` – поведение stencil test'а для front-facing и back-facing граней
- Сам stencil test имеет вид $f(stencil, ref)$, где
  - *stencil* – текущее значение в stencil буфере

Stencil тест

- Stencil test настраивается в `render_pipeline_descriptor.depth_stencil.stencil`
  - `read_mask`, `write_mask` – побитовые маски, которые будут применены к значениям в stencil buffer'е при его чтении/записи
  - `front`, `back` – поведение stencil test'а для front-facing и back-facing граней
- Сам stencil test имеет вид $f(stencil, ref)$, где
  - *stencil* – текущее значение в stencil буфере
  - *ref* – значение, выставляемое через `render_pass.set_stencil_reference()`

Stencil тест

- Stencil test настраивается в `render_pipeline_descriptor.depth_stencil.stencil`
  - `read_mask`, `write_mask` – побитовые маски, которые будут применены к значениям в stencil buffer'е при его чтении/записи
  - `front`, `back` – поведение stencil test'а для front-facing и back-facing граней
- Сам stencil test имеет вид $f(stencil, ref)$, где
  - *stencil* – текущее значение в stencil буфере
  - *ref* – значение, выставляемое через `render_pass.set_stencil_reference()`
  - f – настраиваемая функция

Stencil тест

- Настройка функции для stencil test'а:
  `stencil.{front/back}.compare` – те же функции, что и для теста глубины: `Always`, `Less`, и т.п.

Stencil тест

- Настройка функции для stencil test'а:
  `stencil.{front/back}.compare` – те же функции, что и для теста глубины: `Always`, `Less`, и т.п.
- Если stencil test не прошёл, пиксель не нарисуется

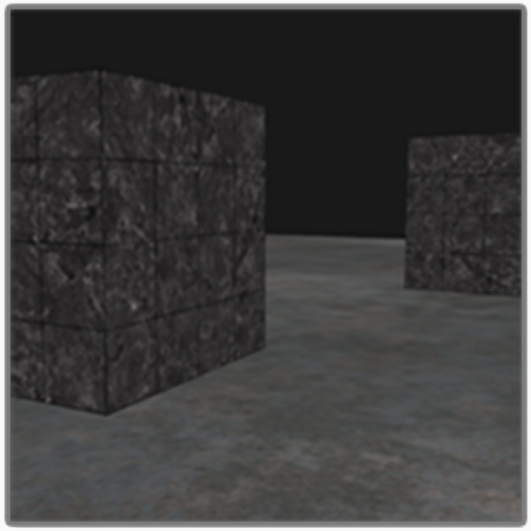
Stencil тест

- Настройка функции для stencil test'а:
  `stencil.{front/back}.compare` – те же функции, что и для теста глубины: `Always`, `Less`, и т.п.
- Если stencil test не прошёл, пиксель не нарисуется
- Настройка того, что произойдёт со значениями в stencil буфере – *stencil operation*:
  - `fail_op` – если не прошёл stencil тест
  - `depth_fail_op` – если прошёл stencil тест, но не прошёл depth тест
  - `pass_op` – если прошли оба теста

Stencil тест

- Возможные значения *stencil operation*:
  - `Keep`
  - `Zero`
  - `Replace`
  - `Invert`
  - `IncrementClamp`
  - `DecrementClamp`
  - `IncrementWrap`
  - `DecrementWrap`

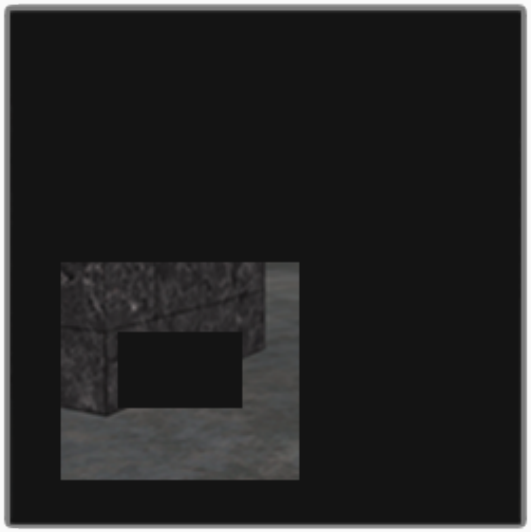
Stencil тест



Color buffer

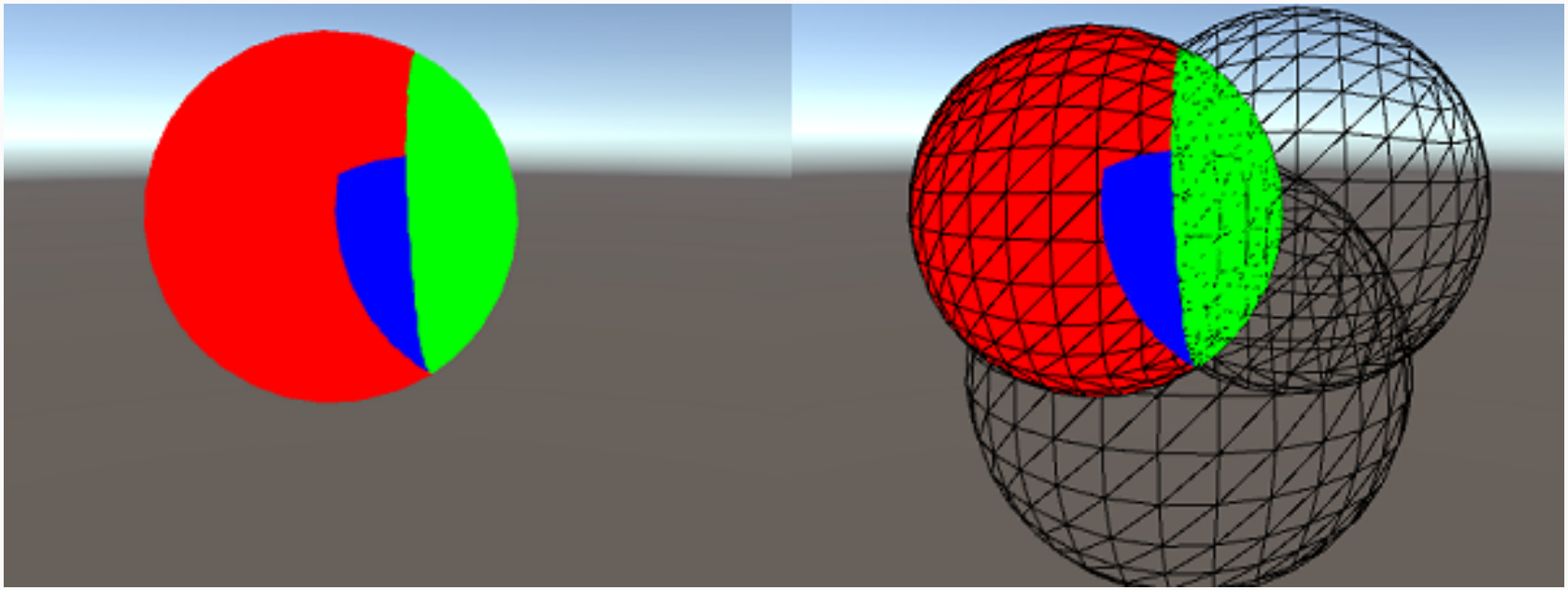


Stencil buffer

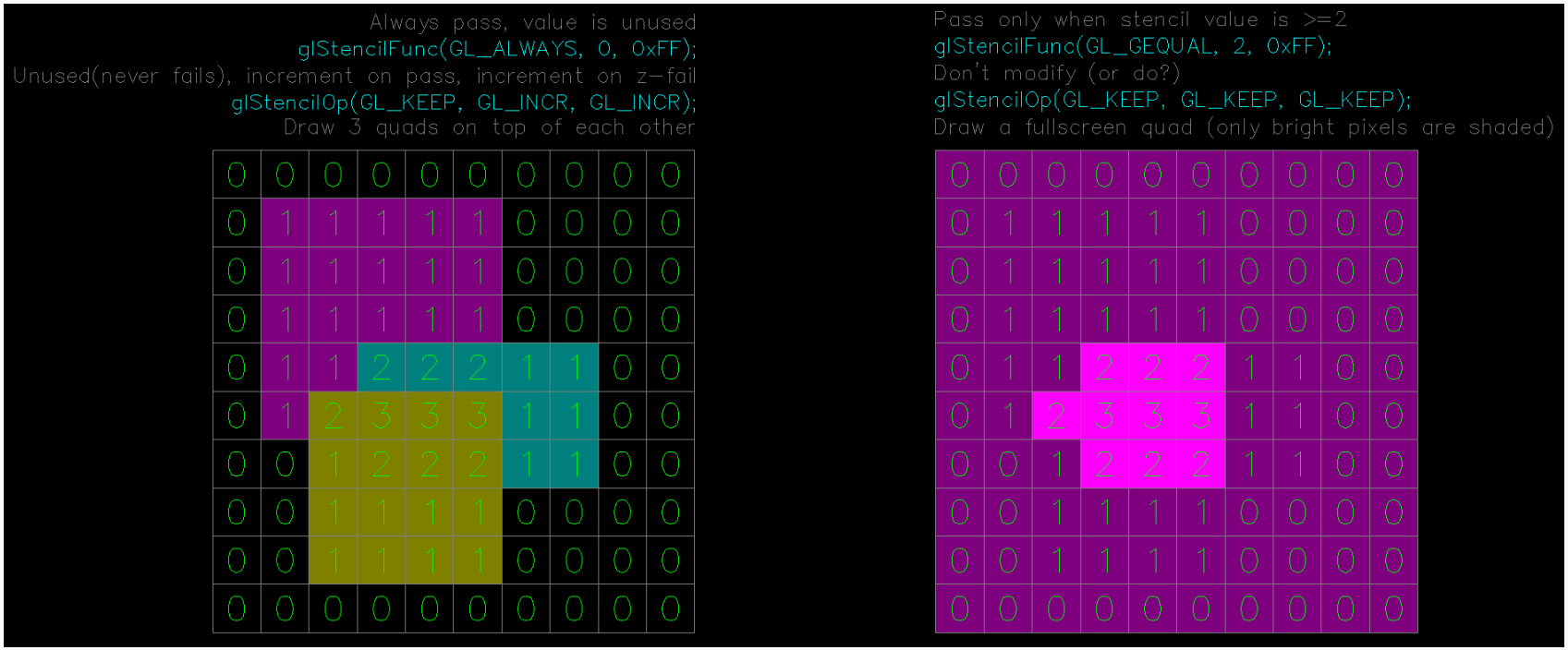


After stencil test

Stencil тест



Stencil тест



Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)

Shadow volumes (stencil shadows)



Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)

Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)
- Любая ситуация, в которой нужно ограничить рисование определённых пикселей:

Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)
- Любая ситуация, в которой нужно ограничить рисование определённых пикселей:
  - Симулятор самолёта: сначала рисуется внутренность самолёта, затем – окружающий мир, только там, где не был нарисован самолёт $\Longrightarrow$ можно избежать проблем с точностью буфера глубины

Microsoft flight simulator

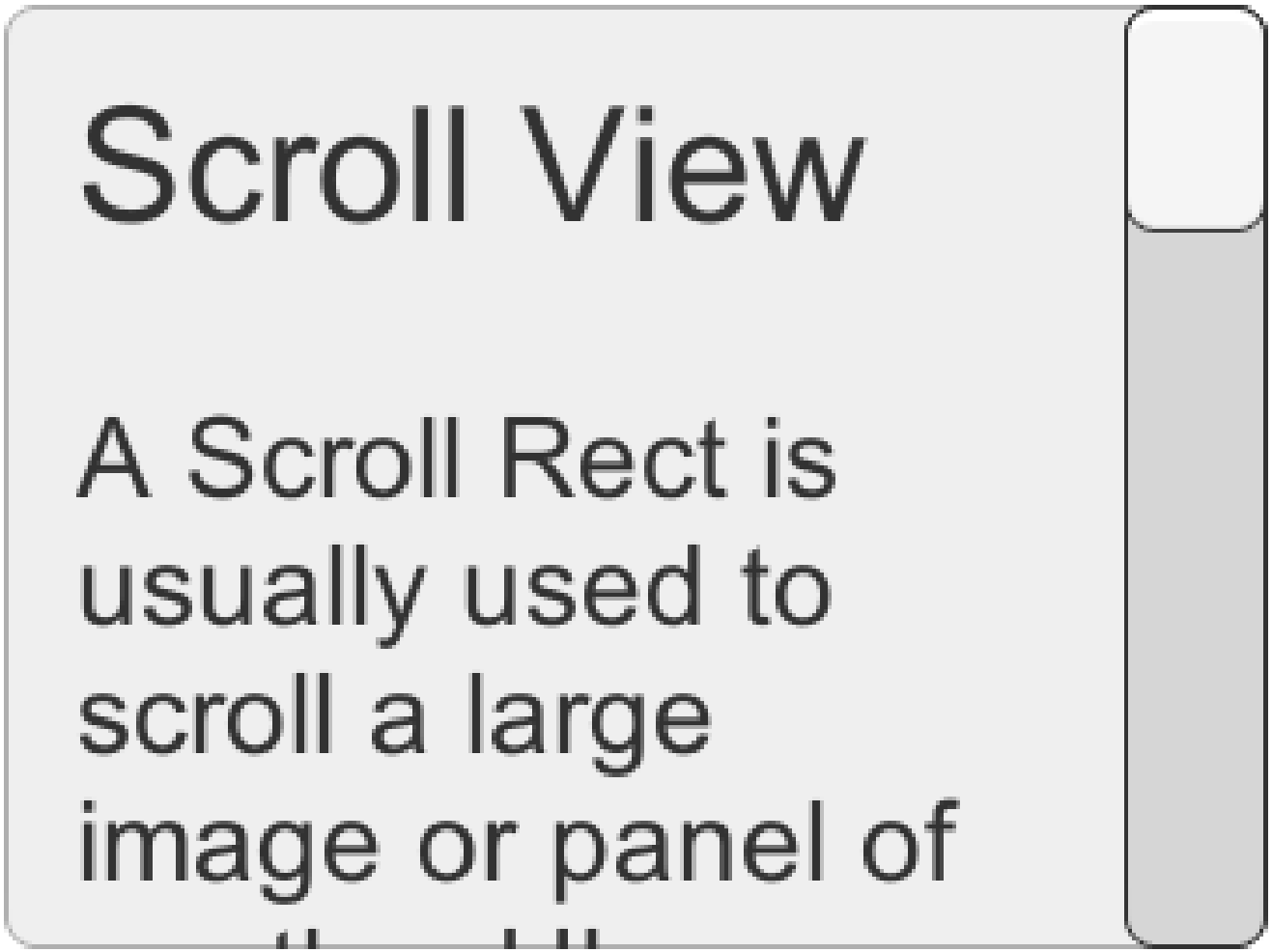


Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)
- Любая ситуация, в которой нужно ограничить рисование определённых пикселей:
  - Симулятор самолёта: сначала рисуется внутренность самолёта, затем – окружающий мир, только там, где не был нарисован самолёт $\Longrightarrow$ можно избежать проблем с точностью буфера глубины

Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)
- Любая ситуация, в которой нужно ограничить рисование определённых пикселей:
  - Симулятор самолёта: сначала рисуется внутренность самолёта, затем – окружающий мир, только там, где не был нарисован самолёт $\Longrightarrow$ можно избежать проблем с точностью буфера глубины
  - UI, который нужно нарисовать в какой-то ограниченной области экрана (например, scroll)


Scroll View
A Scroll Rect is
usually used to
scroll a large
image or panel of

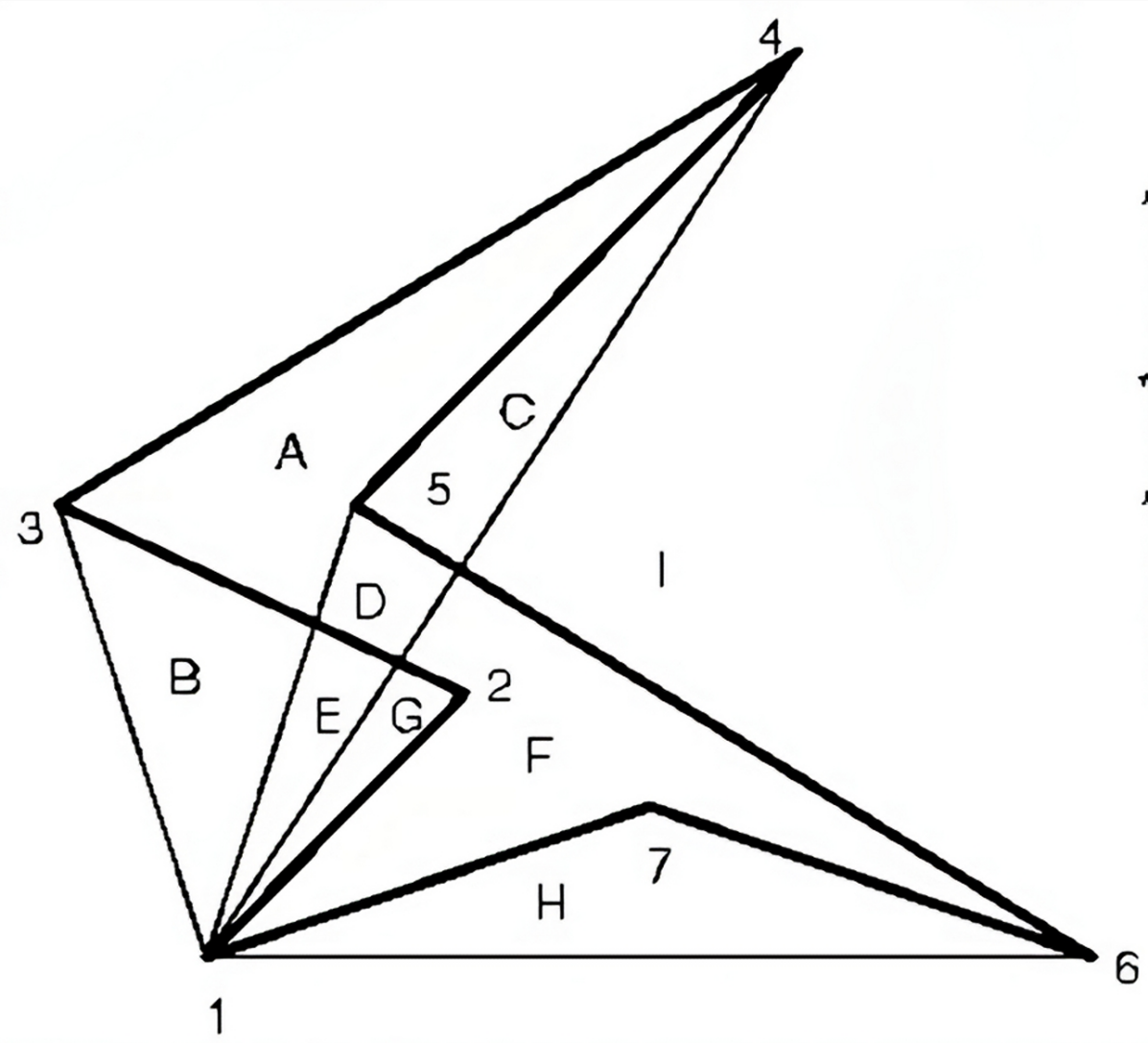
Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)
- Любая ситуация, в которой нужно ограничить рисование определённых пикселей:
  - Симулятор самолёта: сначала рисуется внутренность самолёта, затем – окружающий мир, только там, где не был нарисован самолёт $\Longrightarrow$ можно избежать проблем с точностью буфера глубины
  - UI, который нужно нарисовать в какой-то ограниченной области экрана (например, scroll)

Stencil буфер: применение

- Некоторые алгоритмы рисования теней (stencil shadows / shadow volumes)
- Любая ситуация, в которой нужно ограничить рисование определённых пикселей:
  - Симулятор самолёта: сначала рисуется внутренность самолёта, затем – окружающий мир, только там, где не был нарисован самолёт $\Longrightarrow$ можно избежать проблем с точностью буфера глубины
  - UI, который нужно нарисовать в какой-то ограниченной области экрана (например, scroll)
  - Рисование невыпуклых полигонов (odd-even rule)

Невыпуклый полигон



'A: 134
B: 123 134
C: 134 145
"D: 134 145 156
E: 123 134 145 156
'F: 156
G: 123 156
H: 156 167
I: (none)

Stencil буфер: ссылки

- khronos.org/opengl/wiki/Stencil_Test
- learnopengl.com/Advanced-OpenGL/Stencil-testing
- open.gl/depthstencils
- en.wikibooks.org/wiki/OpenGL_Programming/Stencil_buffer

Render to texture

- При описании render pass'а мы указываем, куда (в какой texture view) этот render pass будет рисовать

Render to texture

- При описании render pass’а мы указываем, куда (в какой texture view) этот render pass будет рисовать
- До этого мы передавали в него texture view, полученный из surface texture, чтобы рисовать на экран

Render to texture

- При описании render pass'а мы указываем, куда (в какой texture view) этот render pass будет рисовать
- До этого мы передавали в него texture view, полученный из surface texture, чтобы рисовать на экран
- Туда можно передать и любой другой texture view, полученный из любой текстуры

Render to texture

- При описании render pass'а мы указываем, куда (в какой texture view) этот render pass будет рисовать
- До этого мы передавали в него texture view, полученный из surface texture, чтобы рисовать на экран
- Туда можно передать и любой другой texture view, полученный из любой текстуры
- Приём рендеринга в текстуру для дальнейшего её использования называется *render to texture*

Render to texture

- `render_pass.color_attachment` содержит массив attachment'ов, каждый из которых ссылается на конкретную texture view

Render to texture

- `render_pass.color_attachment` содержит массив attachment'ов, каждый из которых ссылается на конкретную texture view
- Этому массиву соответствуют индексы в `@location(0)` возвращаемого значения фрагментного шейдера

Render to texture

- `render_pass.color_attachment` содержит массив attachment'ов, каждый из которых ссылается на конкретную texture view
- Этому массиву соответствуют индексы в `@location(0)` возвращаемого значения фрагментного шейдера
- Текстуры, в которую совершается рендеринг, должны быть созданы с флагом `RenderAttachment`

Render to texture

- `render_pass.color_attachment` содержит массив attachment'ов, каждый из которых ссылается на конкретную texture view
- Этому массиву соответствуют индексы в `@location(0)` возвращаемого значения фрагментного шейдера
- Текстуры, в которую совершается рендеринг, должны быть созданы с флагом `RenderAttachment`
- Все attachment'ы должны быть одного размера

Render to texture

- `render_pass.color_attachment` содержит массив attachment'ов, каждый из которых ссылается на конкретную texture view
- Этому массиву соответствуют индексы в `@location(0)` возвращаемого значения фрагментного шейдера
- Текстуры, в которую совершается рендеринг, должны быть созданы с флагом `RenderAttachment`
- Все attachment'ы должны быть одного размера
- Каждому attachment'у можно настроить своё поведение в начале и конце render pass'а

Color attachment: load & store

- `attachment.ops.load` настраивает поведение **при старте** render pass'а:

Color attachment: load & store

- `attachment.ops.load` настраивает поведение **при старте** render pass'а:
  - `Clear`: очистить фиксированным цветом (свой цвет для каждого attachment'а)
  - `Load`: загрузить содержимое attachment'а (т.е. соответствующей текстуры)
  - `DontCare`: нам не важно содержимое на начало render pass'а (скорее всего, мы перерисуем все пиксели сами)

Color attachment: load & store

- `attachment.ops.load` настраивает поведение **при старте** render pass'а:
  - `Clear`: очистить фиксированным цветом (свой цвет для каждого attachment'а)
  - `Load`: загрузить содержимое attachment'а (т.е. соответствующей текстуры)
  - `DontCare`: нам не важно содержимое на начало render pass'а (скорее всего, мы перерисуем все пиксели сами)
- `attachment.ops.store` настраивает поведение **в конце** render pass'а:

Color attachment: load & store

- `attachment.ops.load` настраивает поведение **при старте** render pass'а:
  - `Clear`: очистить фиксированным цветом (свой цвет для каждого attachment'а)
  - `Load`: загрузить содержимое attachment'а (т.е. соответствующей текстуры)
  - `DontCare`: нам не важно содержимое на начало render pass'а (скорее всего, мы перерисуем все пиксели сами)
- `attachment.ops.store` настраивает поведение **в конце** render pass'а:
  - `Store`: сохранить результат рендеринга в attachment (т.е. в соответствующую текстуру)
  - `Discard`: нам не нужно содержимое attachment'а после этого render pass'а

Transient attachment

- Есть особая архитектура GPU, применяющаяся в мобильных устройствах (Arm Mali, Qualcomm Adreno, PowerVR) или некоторых ноутбуках (Apple Silicon): *майловые (tile-based) GPU*

Transient attachment

- Есть особая архитектура GPU, применяющаяся в мобильных устройствах (Arm Mali, Qualcomm Adreno, PowerVR) или некоторых ноутбуках (Apple Silicon): *майловые (tile-based) GPU*
- Они делят весь экран на *майлы* (например, 16x16 пикселей), запоминают список треугольников попавших в каждый тайл, и затем растеризуют каждый тайл по-отдельности в быструю локальную (*on-chip*) память

Transient attachment

- Есть особая архитектура GPU, применяющаяся в мобильных устройствах (Arm Mali, Qualcomm Adreno, PowerVR) или некоторых ноутбуках (Apple Silicon): *майловые (tile-based) GPU*
- Они делят весь экран на *майлы* (например, 16x16 пикселей), запоминают список треугольников попавших в каждый тайл, и затем растеризуют каждый тайл по-отдельности в быструю локальную (*on-chip*) память
- Это может быть медленнее, чем типичная растеризация на десктопной GPU, но расходует меньше электричества

Transient attachment

- Есть особая архитектура GPU, применяющаяся в мобильных устройствах (Arm Mali, Qualcomm Adreno, PowerVR) или некоторых ноутбуках (Apple Silicon): *майловые (tile-based) GPU*
- Они делят весь экран на *майлы* (например, 16x16 пикселей), запоминают список треугольников попавших в каждый тайл, и затем растеризуют каждый тайл по-отдельности в быструю локальную (*on-chip*) память
- Это может быть медленнее, чем типичная растеризация на десктопной GPU, но расходует меньше электричества
- Текстура, созданная с флагом `TransientAttachment` и использующая `load = Clear/DontCare` и `store = Discard` может быть оптимизирована на таких GPU: её можно вообще не хранить в видеопамяти, только on-chip

Render to texture

- В качестве attachment'а можно указать конкретный слой 2D array текстуры, или конкретную грань cubemap текстуры: для этого нужно указать `base_array_layer` при создании texture view

Render to texture

- В качестве attachment'а можно указать конкретный слой 2D array текстуры, или конкретную грань cubemap текстуры: для этого нужно указать `base_array_layer` при создании texture view
- В качестве attachment'а можно указать срез 3D текстуры с фиксированным значением координаты Z: для этого нужно указать `depth_slice` в самом color attachment'е

Render to texture: форматы

- Форматы пикселей, в которые можно совершать рендеринг, называются *renderable formats*

Render to texture: форматы

- Форматы пикселей, в которые можно совершать рендеринг, называются *renderable formats*
- Не все форматы являются *renderable*; гарантируется поддержка

Render to texture: форматы

- Форматы пикселей, в которые можно совершать рендеринг, называются *renderable formats*
- Не все форматы являются *renderable*; гарантируется поддержка
  - 8-битных нормированных (напр. `RGBA8Unorm`)
  - 16-битных и 32-битных floating-point (напр. `RG16Float`)
  - Всех целочисленных форматов (напр. `RG32Uint`)
  - 10-10-10-2 форматов (`RGB10A2Unorm`, `RGB10A2Uint`)

Render to texture: форматы

- Форматы пикселей, в которые можно совершать рендеринг, называются *renderable formats*
- Не все форматы являются *renderable*; гарантируется поддержка
  - 8-битных нормированных (напр. `RGBA8Unorm`)
  - 16-битных и 32-битных floating-point (напр. `RG16Float`)
  - Всех целочисленных форматов (напр. `RG32Uint`)
  - 10-10-10-2 форматов (`RGB10A2Unorm`, `RGB10A2Uint`)
- Другие форматы тоже могут поддерживаться, это нужно запрашивать в `required_features` при создании `Device`

Render to texture: форматы

- Форматы пикселей, в которые можно совершать рендеринг, называются *renderable formats*
- Не все форматы являются *renderable*; гарантируется поддержка
  - 8-битных нормированных (напр. `RGBA8Unorm`)
  - 16-битных и 32-битных floating-point (напр. `RG16Float`)
  - Всех целочисленных форматов (напр. `RG32Uint`)
  - 10-10-10-2 форматов (`RGB10A2Unorm`, `RGB10A2Uint`)
- Другие форматы тоже могут поддерживаться, это нужно запрашивать в `required_features` при создании `Device`
  - Например, `RG11B10Float` полезен для HDR-рендеринга

Render to texture: форматы

- Форматы пикселей, в которые можно совершать рендеринг, называются *renderable formats*
- Не все форматы являются *renderable*; гарантируется поддержка
  - 8-битных нормированных (напр. `RGBA8Unorm`)
  - 16-битных и 32-битных floating-point (напр. `RG16Float`)
  - Всех целочисленных форматов (напр. `RG32Uint`)
  - 10-10-10-2 форматов (`RGB10A2Unorm`, `RGB10A2Uint`)
- Другие форматы тоже могут поддерживаться, это нужно запрашивать в `required_features` при создании `Device`
  - Например, `RG11B10Float` полезен для HDR-рендеринга
- Формат может быть renderable но не поддерживать blending (например, целочисленные форматы)

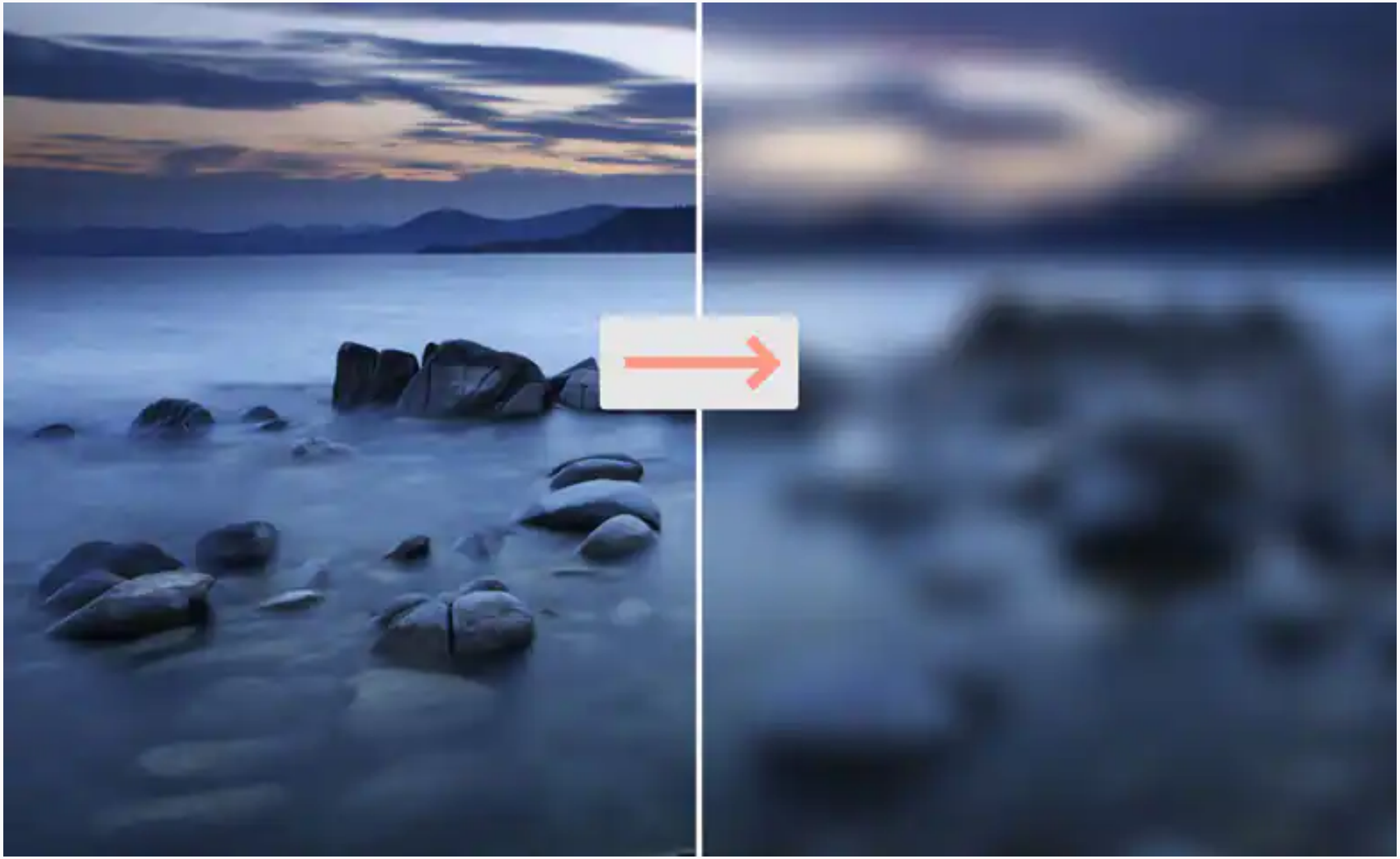
Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
    - Размытие

Размытие



Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)

Свечение



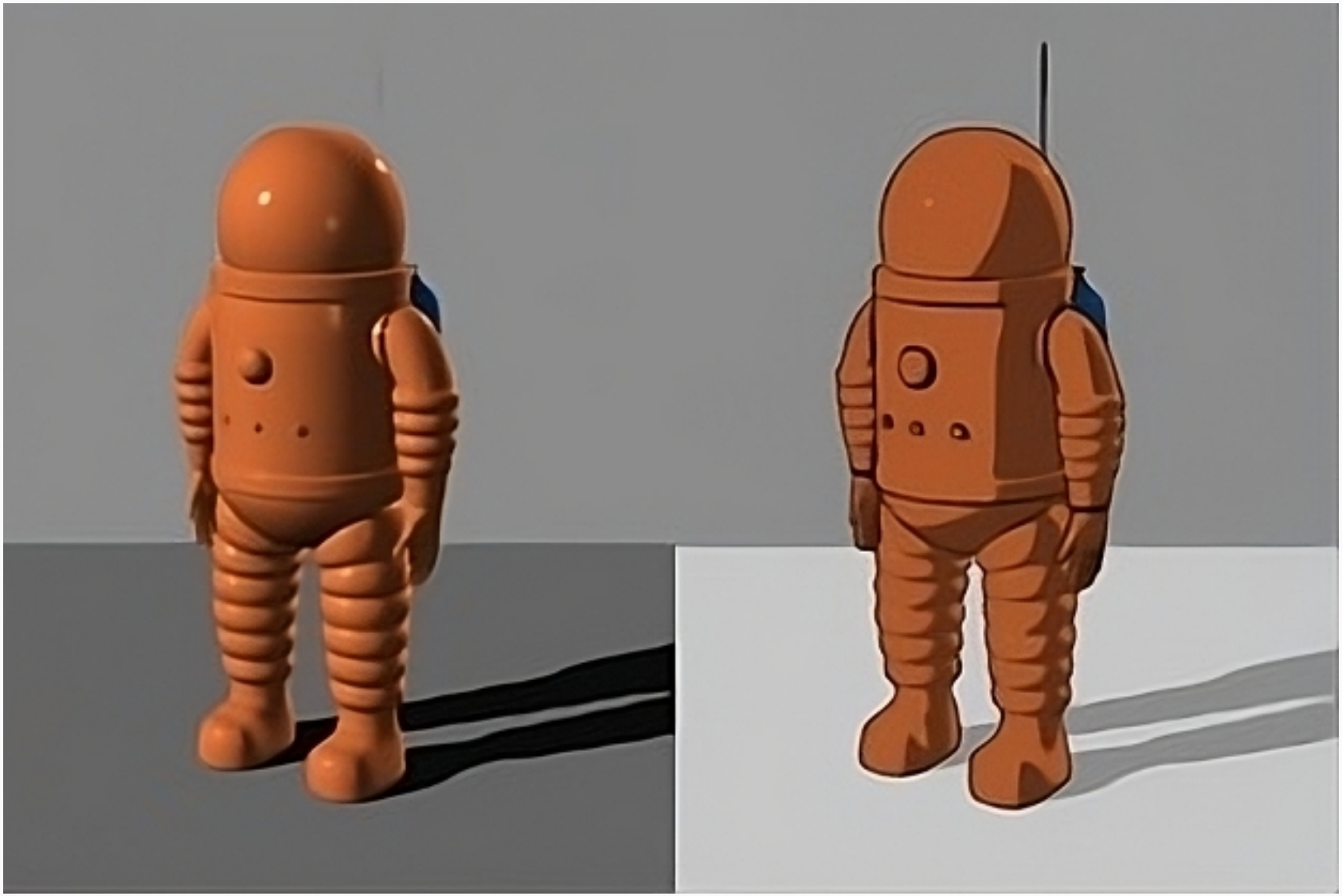
Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)

Toon shading



Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)

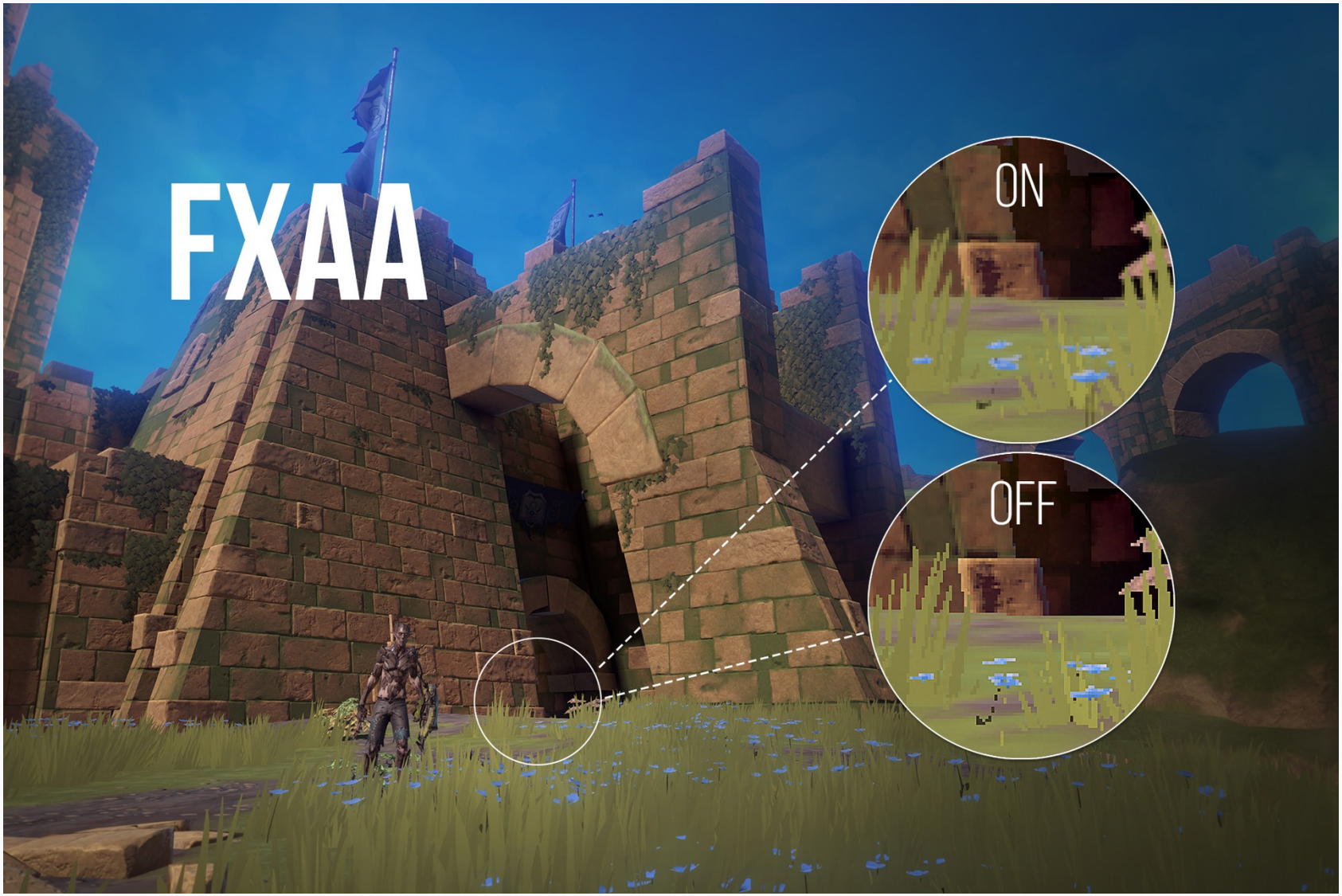
Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)

FXAA



Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)

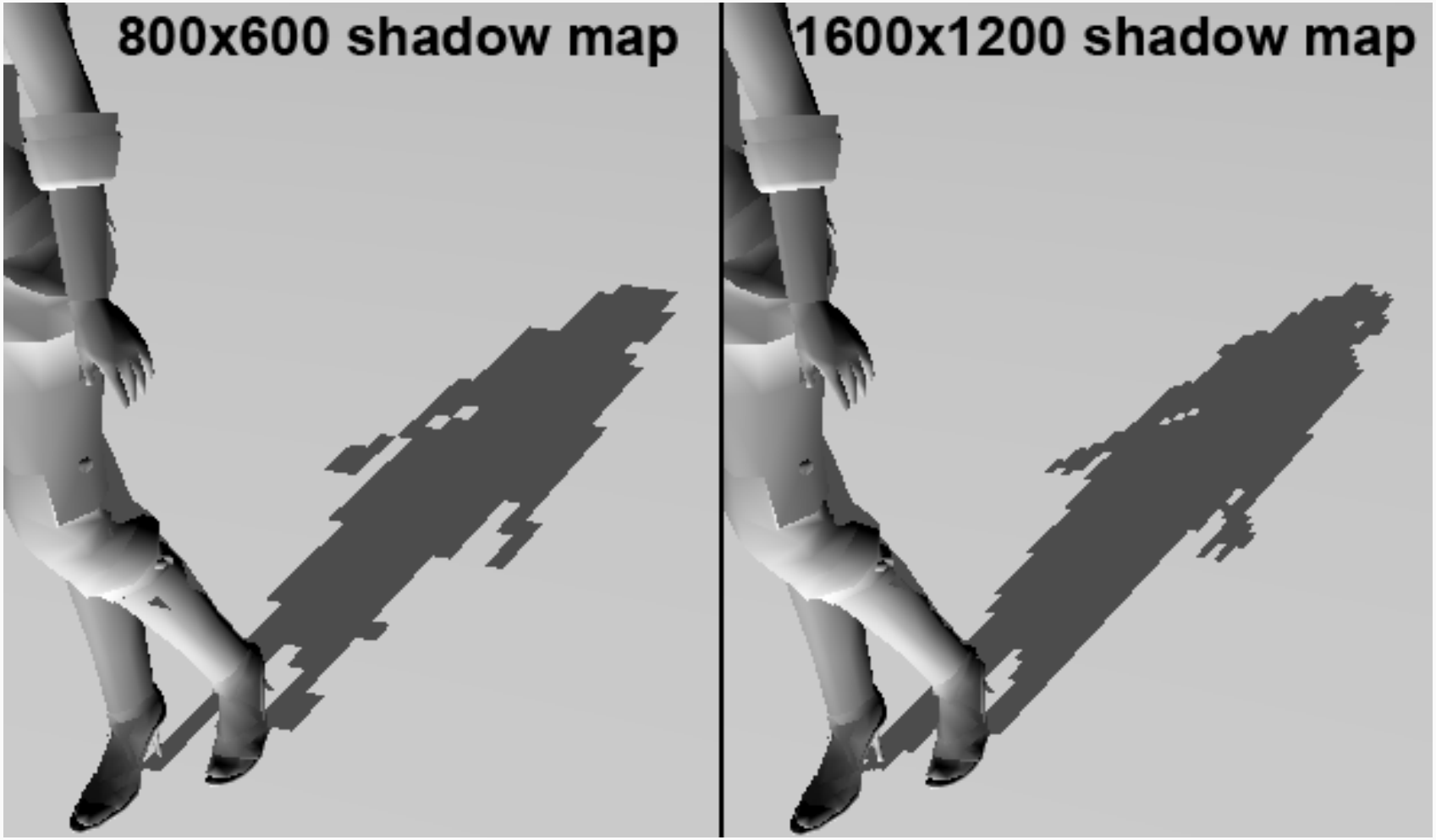
Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)
  - И т.д.

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)
  - И т.д.
- Тени (shadow maps)

Shadow mapping



Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)
  - И т.д.
- Тени (shadow maps)

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)
  - И т.д.
- Тени (shadow maps)
- Отражения

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)
  - И т.д.
- Тени (shadow maps)
- Отражения
- Deferred shading

Примеры использования render to texture

- Пост-обработка кадра – реализуется рисованием кадра в текстуру, и затем рисованием полноэкранного прямоугольника на экран с нужным шейдером, читающим эту текстуру
  - Размытие
  - Свечение (bloom)
  - Toon shading (edge detection, color grading)
  - HDR, гамма-коррекция
  - Сглаживание (FXAA)
  - И т.д.
- Тени (shadow maps)
- Отражения
- Deferred shading
- И очень много другого

Размытие

- Усреднение значений соседних пикселей в некоторой окрестности

Размытие

- Усреднение значений соседних пикселей в некоторой окрестности
- Box blur: все веса одинаковые, получается угловатым

Размытие

- Усреднение значений соседних пикселей в некоторой окрестности
- Box blur: все веса одинаковые, получается угловатым
- Gaussian blur: веса пропорциональны гауссиане $\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$, получается равномерное сглаживание

Размытие

- Несколько итераций box blur похожи на один gaussian blur (центральная предельная теорема)

Размытие

- Несколько итераций box blur похожи на один gaussian blur (центральная предельная теорема)
- Обычно gaussian blur делают в два прохода: один размывает по горизонтали, второй – по вертикали (*separable blur*)

Размытие

- Несколько итераций box blur похожи на один gaussian blur (центральная предельная теорема)
- Обычно gaussian blur делают в два прохода: один размывает по горизонтали, второй – по вертикали (*separable blur*)
- Можно использовать linear filtering и подбирать координаты sampling'а текстуры так, чтобы он автоматически давал правильные коэффициенты размытия

Размытие

- Несколько итераций box blur похожи на один gaussian blur (центральная предельная теорема)
- Обычно gaussian blur делают в два прохода: один размывает по горизонтали, второй – по вертикали (*separable blur*)
- Можно использовать linear filtering и подбирать координаты sampling'а текстуры так, чтобы он автоматически давал правильные коэффициенты размытия
- Можно комбинировать размытие с уменьшением/увеличением (*downsampling/upsampling*) изображения

Размытие

- Несколько итераций box blur похожи на один gaussian blur (центральная предельная теорема)
- Обычно gaussian blur делают в два прохода: один размывает по горизонтали, второй – по вертикали (*separable blur*)
- Можно использовать linear filtering и подбирать координаты sampling'а текстуры так, чтобы он автоматически давал правильные коэффициенты размытия
- Можно комбинировать размытие с уменьшением/увеличением (*downsampling/upsampling*) изображения
- Самые эффективные современные алгоритмы комбинируют все эти идеи (см. *Dual Kawase Blur*)

Box blur

```
@group(0) @location(0) input_texture: texture_2d<f32>;
@group(0) @location(1) nearest_sampler: sampler;

@fragment
fn fragmentMain(in: VertexOut) -> @location(0) vec4f {
    const N = 5;
    var sum = vec4f(0.0);
    let size = vec2f(textureDimensions(input_texture, 0));

    for (var x = -N; x <= N; x += 1) {
        for (var y = -N; y <= N; y += 1) {
            let offset = vec2f(vec2i(x, y));
            sum += textureSampleLevel(input_texture, nearest_sampler,
              in.texcoord + offset / size, 0);
        }
    }

    return sum / f32((2 * N + 1) * (2 * N + 1));
}
```

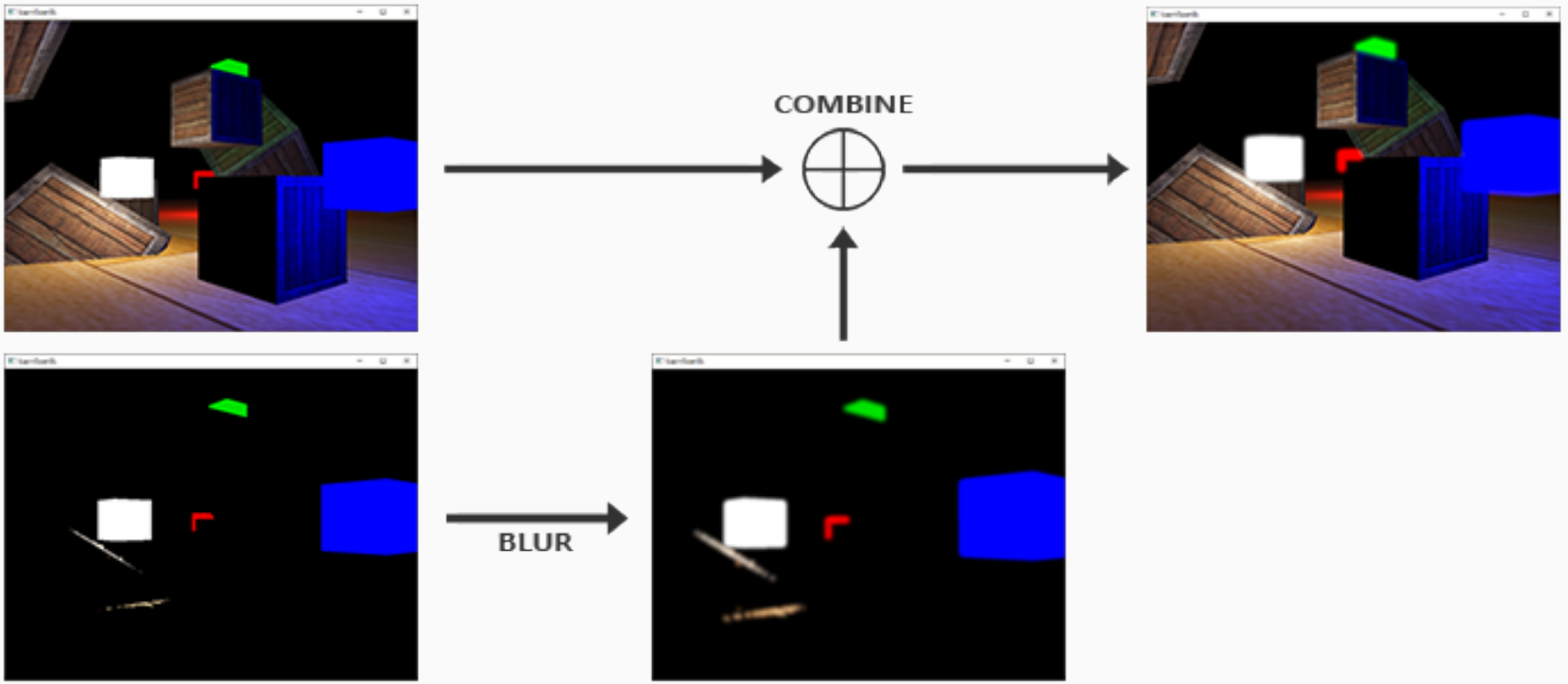
Gaussian blur

```
@fragment
fn fragmentMain(in: VertexOut) -> @location(0) vec4f {
    const N = 5;
    const sigma = 2.0;
    var sum = vec4f(0.0);
    var sum_weights = 0.0;
    let size = vec2f(textureDimensions(input_texture, 0));

    for (var x = -N; x <= N; x += 1) {
        for (var y = -N; y <= N; y += 1) {
            let offset = vec2f(vec2i(x, y));
            let weight = exp(- dot(offset, offset)
              / 2.0 / (sigma * sigma));
            sum += textureSampleLevel(input_texture, nearest_sampler,
              in.texcoord + offset / size, 0) * weight;
            sum_weights += weight;
        }
    }

    return sum / sum_weights;
}
```

Bloom



Toon shading (cel shading)

- Edge detection + color grading

Toon shading (cel shading)

- Edge detection + color grading
- Edge detection: шейдер находит и выделяет резкие перепады цвета или глубины (используя, например, *Sobel filter*)

Toon shading (cel shading)

- Edge detection + color grading
- Edge detection: шейдер находит и выделяет резкие перепады цвета или глубины (используя, например, *Sobel filter*)
- Color grading: палитра цветов искусственно уменьшается (например, сжимается до 2-3 бит на канал)

Color grading

```
@fragment
fn fragmentMain(in: VertexOut) -> @location(0) vec4f {
    let input = textureSampleLevel(input_texture, nearest_sampler,
      in.texcoord, 0);

    return floor(input * 4.0) / 3.0;
}
```

Ссылки

- learnopengl.com/Advanced-Lighting/Bloom
- Gaussian blur (youtube video tutorial)
- How to blur images at 20000 FPS (or 50 microseconds)